UNIVERSITY POLITEHNICA OF BUCHAREST
FACULTY OF AUTOMATIC CONTROL AND COMPUTERS
COMPUTER SCIENCE AND ENGINEERING DEPARTMENT





THESIS PROJECT

Tasks Scheduling in Distributed Systems

Bogdan Dumitrescu

Thesis advisor:

Prof. dr. Radu-Ioan Ciobanu

BUCHAREST

2026

ABSTRACT

Modern data-processing workflows often run on clusters that are small, mixed and imperfect: nodes differ in CPU type, memory, storage speed, processor architecture, cooling behaviour and network position. The default Kubernetes scheduler mainly checks whether a Pod fits on a node, which is not enough when the same workflow contains CPU-bound, memory-bound and I/O-bound tasks whose performance depends on where their data, images and previous runs are located. This project proposes a task scheduler for Kubernetes that treats workflow placement as a measured decision rather than a simple fit test.

The scheduler combines several relevant fields in one practical policy: resource requests, node type, architecture compatibility, learned execution time, startup cost, data locality, live bandwidth, thermal headroom, recent failures, task priority, workflow fairness and task position inside the DAG. These signals are used to estimate the effective completion time of each possible placement and to keep learning when some task-node pairs are still under-sampled. The system is implemented with Kubernetes Custom Resources, an in-cluster controller, a custom scheduler and lightweight DaemonSets for the data plane, bandwidth probing and thermal telemetry. The result is a deployable prototype that preserves the usual Kubernetes workflow while making scheduling decisions that are aware of the actual conditions of a heterogeneous cluster.

CONTENTS

1 Introduction **3**

1.1 Subject and context . . . 3

1.2 Motivation . . . 3

1.3 Objectives . . . 4

1.4 Scope and expected impact . . . 4

2 State of the art **5**

3 Project model **6**

3.1 High-level architecture . . . 6

3.2 The user contract: Workflow and TaskTemplate CRDs . . . 6

3.3 Two-tier scheduling: from workflows to tasks . . . 7

3.4 Scoring function and ECT . . . 8

3.5 Live signals . . . 8

3.6 Producer-local data plane . . . 8

3.7 Profile store and learning model . . . 9

3.8 Relevance of the modelled factors . . . 9

3.9 Baselines and why they fail on heterogeneous clusters . . . 11

3.10 Design-choice rationale . . . 12

4 Implementation journey **13**

4.1 Iteration 0 to 1: simulator and legacy live scheduler . . . 13

4.2 Iteration 2: pluggable scheduling policy . . . 13

4.3 Iteration 3: producer-local data plane . . . 13

4.4 Iteration 4: CRDs and controller . . . 13

4.5 Iteration 5 to 6: telemetry and adaptive scheduling . . . 14
4.6 Cross-cutting engineering . . . 14

5 Research methodology **15**
5.1 Two execution surfaces, one scheduler model . . . 15
5.2 Experimental topology . . . 15
5.3 Workload and comparison protocol . . . 16

6 Contributions **17**
6.1 Theoretical . . . 17
6.2 Practical and engineering . . . 17

7 Performance analysis metrics **18**

8 Preliminary results and analysis **19**
8.1 Pilot setup and end-to-end validation . . . 19
8.2 Specialisation behaviour . . . 19
8.3 Indicative policy comparison . . . 20

9 Operator's guide and convergence expectations **21**
9.1 Prerequisites and installation . . . 21
9.2 Task templates and workflows . . . 21
9.3 Convergence and remediation . . . 22

10 Conclusion and further work **23**
10.1 Summary . . . 23
10.2 Further work . . . 23

1 INTRODUCTION

1.1 Subject and context

Workflow scheduling on Kubernetes is dominated by the cluster's default scheduler (`kube-scheduler`), which optimises for resource fit and basic affinity but treats nodes as essentially interchangeable. That assumption holds in the large, homogeneous clusters Kubernetes was originally designed for, but breaks badly on the cluster this project targets: a small, heterogeneous deployment built from whatever hardware the operator already owns.

1.2 Motivation

A workflow scheduler that wants to do better than `kube-scheduler` on real heterogeneous clusters has to reason about a long list of factors that the default scheduler simply cannot express, and that the published literature usually treats one at a time: *node specialisation* (CPU- vs memory- vs I/O-optimised), *CPU architecture* (`amd64` vs `arm64`, so image `nodeSelectors` and binary compatibility matter), *image warmth / startup cost* (a cold pull on a slow link is a non-trivial fraction of a short task's wall-clock), *data locality* (a parent's output already on disk is free; fetching it from another node costs real seconds), *inter-node bandwidth* (not uniform: a Wi-Fi link to an arm64 box is not a 10 Gb backplane), *thermal headroom and cooling class* (the same chip in a passively-cooled mini-PC throttles long before its tower twin does), *transient failure history* (a node that just dropped two Pods should not be the first pick for the next one), *DAG structure* (critical-path tasks need to go first; fan-out children need their parents' outputs co-located when possible), and *fairness across concurrent workflows* (a long batch job should not starve a short interactive one).

ECT-style estimation, multi-armed-bandit exploration (UCB), critical-path ordering, thermal awareness and gang/preemption logic have each been studied individually, but are rarely combined, almost never integrated cleanly into Kubernetes' control loop, and usually evaluated on simulated *homogeneous-but-labelled* clusters rather than on mixed local hardware. This project insists on integrating all of them in one scoring function J and validating on real heterogeneous hardware. Another problem is storage. Many workflow systems assume shared storage such as Lustre, NFS or S3, but a small local cluster often does not have that. In this project, each node keeps the outputs it produced and serves them over HTTP when another node needs them. The same path is used for bandwidth checks, so the scheduler measures the network path that the tasks actually use.

1.3 Objectives

O1. Design a workflow-aware scheduling model that captures node heterogeneity (CPU/memory/I/O profile, cooling class, architecture) and DAG structure (upward rank, critical path).
O2. Define an effective-completion-time estimator $\widehat{\mathrm{ECT}}(\tau, n \mid A)$ that accounts for the current partial assignment A (parent placements, data sizes).
O3. Combine ECT with a UCB exploration policy and a thermal-headroom term inside a single scoring function J.
O4. Replace shared-filesystem assumptions with a producer-local data plane.
O5. Implement everything as a clean Kubernetes deployment (Workflow CRD, controller, custom scheduler, three DaemonSets) that coexists with any other workload.
O6. Validate end-to-end on real Kubernetes (`kind` for CI and a two-host `k3s` cluster for larger experiments), and maintain a regression simulator running the same decision model.

1.4 Scope and expected impact

Cross-cluster federation, live preemption / checkpoint-restore, speculative execution and validation on external benchmark suites are outside the current scope. The implementation includes hooks for some of these directions, but the live binder runs in *place-and-let-finish* mode for now. The current validation uses project-defined workflows built from CPU-bound, memory-bound and I/O-bound task templates, with linear and fan-out / fan-in DAG shapes. It does not use Pegasus DAX, CyberShake or other published workflow benchmark datasets.

If the proposed policy is shown to consistently reduce workflow makespan or improve fairness on heterogeneous clusters relative to the default scheduler, without intrusive changes to user workloads, it offers a directly deployable improvement to a very widely used piece of cluster software. Even a partial result is useful: it clarifies which of the measured signals, such as ECT, UCB, bandwidth, locality and thermal state, carry weight in practice and which can be simplified later.

2 STATE OF THE ART

Kubernetes scheduling inherits ideas from large cluster managers such as Borg [3] and Omega [4]: a scheduler observes pending work, evaluates the cluster state and binds work to machines. These systems are highly influential, but their assumptions are closer to managed datacentres than to small mixed clusters assembled from different hosts.

DAG scheduling research adds another useful idea: tasks on the critical path should not be delayed behind less important work. TaskScheduler does not use HEFT [1] as its scheduling policy, because workflow priority and fairness come first, but it reuses the simpler critical-path signal when choosing between ready tasks.

UCB-style exploration [2] gives a lightweight way to learn from repeated executions without training a large model. Deep reinforcement learning has also been explored for distributed stream processing [5], but this project keeps the policy interpretable: measured runtimes, explicit penalties and controlled exploration. Thermal-aware scheduling is similarly relevant because small clusters often mix passively and actively cooled devices.

Kubernetes-native workflow tools such as Volcano, Kueue and Argo Workflows solve adjacent problems: gang admission, queueing or DAG execution. TaskScheduler instead focuses on placement itself, using workflow structure, learned history, locality, bandwidth and thermal signals in one scheduler.

The homogeneous-cluster assumption. The single most consistent gap in the surveyed literature, from Borg and Omega through to the recent DRL schedulers, is the implicit assumption that the cluster is homogeneous or only *logically* heterogeneous (different price tiers of otherwise identical VMs). Mixed-architecture deployments (`amd64+arm64`), passively vs actively cooled nodes in the same cluster, and the absence of a shared filesystem are simply not considered. This is reasonable for cloud-native research, but the real world of home labs and small-business deployments looks exactly like that. TaskScheduler is in part an exercise in showing that techniques developed for clean datacentre clusters can be adapted to the messy hardware operators actually have lying around.

3 PROJECT MODEL

This chapter describes the architecture of TaskScheduler, the scheduling algorithm, the live signals it consumes, and the rationale for each design choice. It also explains why several obvious baselines are not adequate on heterogeneous clusters.

3.1 High-level architecture

The Task Scheduler runs entirely inside Kubernetes as two Deployments, three DaemonSets, and a pair of Custom Resource Definitions (Figure 1):

1. **Workflow CRD**: a user submits a workflow as a Kubernetes object containing the DAG, the per-task templates, and the workflow-level metadata (priority, preemptable, workflow class).
2. **TaskTemplate CRD**: the operator registers task types once (image, default resource requests, compatible node types, expected output size). Workflows reference templates by name.
3. `ts-controller` **(Deployment).** Watches Workflow CRDs, expands the DAG into Pods, sets `schedulerName=ts-scheduler`, scrapes `__TS_OUTPUT__=` lines from finished pods' logs and injects them as environment variables on downstream pods. Updates `Workflow.status` on every transition.
4. `ts-scheduler` **(Deployment).** Watches Pending Pods whose `schedulerName` is `ts-scheduler`, runs the scheduler once per tick, and binds each Pod to a node via the `pods/binding` subresource.
5. **Data-plane DaemonSet (**`ts-fileserver`**).** Exposes each node's local output directory over HTTP so that downstream tasks running on a different node can fetch parent outputs.
6. **Telemetry DaemonSets.** `ts-bw-probe` measures inter-node bandwidth every 10 min and annotates its own Node with `ts.io/bw-to-<peer>=<MB/s>`. `ts-thermal` samples the host thermal sensor every 30 s and writes `ts.io/cpu-temp-c`.

3.2 The user contract: Workflow and TaskTemplate CRDs

Splitting the user surface into two CRDs separates the *operator* concerns (which task images exist, what they cost) from the *user* concerns (which tasks to compose into a DAG). `TaskTemplate` declares an image, a class (`CPU_BOUND`, `MEMORY_BOUND`, `IO_BOUND`), default

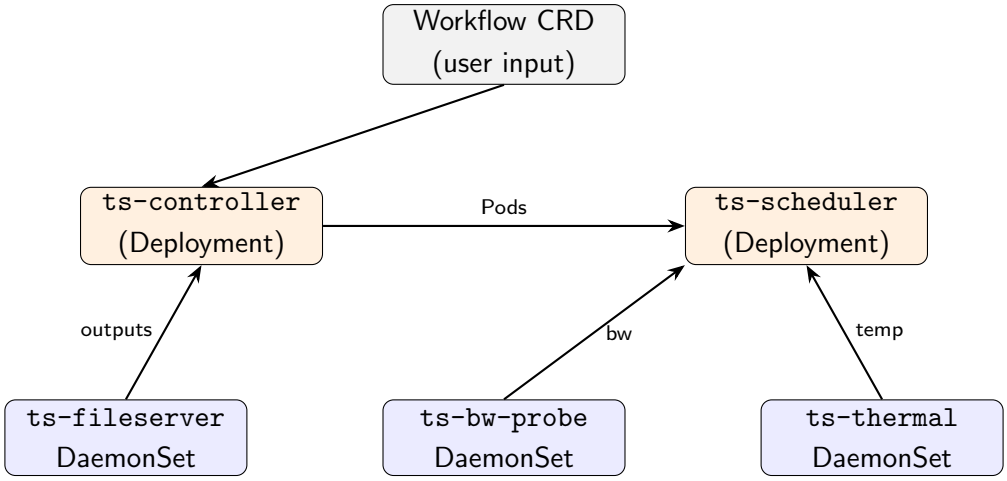


Figure 1: High-level architecture: a thin controller, a custom scheduler, and three DaemonSets supplying data plane and telemetry. All components run as ordinary Kubernetes workloads.

CPU/memory requests, compatible node types, optional `expectedOutputBytes` and an optional `gangGroupId`. `Workflow` declares a priority class and a list of named tasks with `dependsOn` edges that nominate parent task names and the data fields consumed. The controller produces one Pod per task and threads parent outputs through to downstream tasks.

3.3 Two-tier scheduling: from workflows to tasks

The scheduler operates at two levels of granularity:

1. **Workflow level.** Each workflow accumulates virtual runtime v_w, incremented per tick proportional to its current CPU share. The next workflow to dispatch from is the one with the smallest v_w, ensuring long-term fairness across concurrently submitted workflows without strict round-robin rigidity.
2. **Task level.** Within the chosen workflow, the next Pending task is the one with the largest *upward rank*, the longest weighted path from that task to any DAG sink, so that critical-path tasks are placed first.

This decoupling keeps priority and fairness at the workflow level, then uses DAG criticality only after the workflow has been selected. A scheduler that only orders by priority class (as the default Kubernetes scheduler does) would starve low-priority workflows under sustained load; one that only orders by FIFO would let a long critical-path task at the back of the queue dominate makespan.

3.4 Scoring function and ECT

For every ready task, the scheduler first removes nodes that cannot run it because of resource requests or compatibility. For the nodes that remain, it estimates a simple completion time: when the node is free, how long the Pod is likely to take to start, how much parent data must be transferred, and how long this task usually runs on that node.

In plain terms, the base estimate is:

$$\text{estimated finish time} = \text{ready time} + \text{startup time} + \text{transfer time} + \text{runtime}$$

Each part has a direct meaning. Ready time avoids packing too many tasks onto the same node in one scheduling tick. Startup time captures image pulls and other Pod start delays. Transfer time is zero when a child runs on the same node as its data-producing parent, and grows when data has to cross the network. Runtime comes from previous observations of the same task type on the same node, with a safe default when the pair has not been seen before.

After this base estimate, the scheduler adjusts the score with a few practical signals. A node that recently failed this task receives a penalty. A node that is too hot also receives a penalty. Under-sampled task-node pairs receive a small exploration bonus, so the scheduler does not stop learning after the first lucky measurement. The chosen node is the feasible node with the lowest final score.

This is intentionally easier to read than a generic weighted sum. The main number is still measured in seconds, so the operator can understand why a placement was chosen: either it was expected to finish sooner, avoid a transfer, avoid a hot or failing node, or collect a missing measurement.

3.5 Live signals

Two signals are collected continuously by DaemonSets and exposed as Node annotations. `ts-bw-probe` on each node fetches a `_probe.bin` (100 MB) from every peer's `ts-fileserver`, divides size by elapsed time and writes `ts.io/bw-to-<peer>=<MB/s>` every 10 min. The scheduler combines fresh probe values with an EMA of observed real transfers. `ts-thermal` reads the host thermal sensor on Linux (falling back to a load-average proxy on hosts without direct thermal access), every 30 s. Temperature and cooling class are discretised into the four buckets used both to index profile observations and to compute the headroom term in J.

3.6 Producer-local data plane

Each node owns a local output directory mounted into every worker Pod. Same-node parents are read through that mount with no network involvement ("Channel C"); for cross-node

parents ("Channel B"), the controller injects `TS_PARENT_<NAME>_FILESERVER_URL` so the task script can `curl` the parent's output through the producer's `ts-fileserver`. Channel A (metadata-only, via `__TS_OUTPUT__` log lines) is unchanged. No shared filesystem, no object store, and because the data plane and the bandwidth probe share the same HTTP path, predicted and actual transfer cost stay consistent by construction.

3.7 Profile store and learning model

Learning state is keyed by `(task_template_id, node_id)`. Each pair keeps a rolling window of the last 20 successful observations (runtime, startup, CPU/memory ratio at placement, thermal bucket, timestamp), the derived statistics (median, EMA, σ, count, UCB-1), a drift flag (EMA outside $\pm k\sigma$ of the median, triggering re-exploration) and per-node failure counters with exponential decay (half-life 14 h). The store is persisted every $\sim$60 s to Kubernetes storage so learning survives scheduler-pod restarts.

Exploration has three phases: (1) *coverage*, every feasible compatible node sampled at least once per template; (2) *depth*, at least `MIN_OBSERVATIONS_PER_NODE` $= 3$ observations per node; (3) *normal*, 10 % random exploration / 90 % scored exploitation, with the UCB bonus continuing to nudge under-explored or drifting pairs. For k feasible nodes per template this reaches end-of-coverage after $\approx k$ runs and end-of-depth after $\approx 3k$. On the four-node `kind` test cluster the linear three-task workflow leaves coverage after $\approx$4 workflows and reaches depth after $\approx$12. Cold-start placements use the TaskTemplate's compatible-node-types prior and the default-runtime constants in the ECT estimator, so the very first workflow is reasonable, just not yet specialised. The active policy is selected at deployment time, allowing the adaptive and legacy variants to be compared without rebuilding the system.

3.8 Relevance of the modelled factors

The scheduler deliberately collapses a long list of signals into a single seconds-valued score. Each signal earns its place by answering a concrete failure mode of a simpler design; this section lists them and says, for each, why it is in the model.

Node type (CPU_OPT / MEM_OPT / IO_OPT / GENERAL).
The TaskTemplate's `compatibleNodeTypes` is the feasibility filter; without it a CPU-bound task can land on the memory-optimised node and pay a 20 to 40 % per-task slowdown. It also drives the cold-start prior so the very first placement is not random.

CPU architecture (`amd64`, `arm64`).
Images are built multi-arch and the feasibility filter rejects nodes whose architecture does not match the image manifest. Without this term the binder would happily attempt an `ImagePullBackOff` on the wrong arch.

Learned execution time $\mu_{\tau,n,b}$.

Per-template, per-node, per-thermal-bucket median runtime, refined by an EMA and bounded by a rolling window of 20 observations. Two nodes with the same labels can still differ by 30 % in practice; only measurement tells the scheduler this.

Startup time (image warmth).

Median observed Pod `Pending`→`Running` delay for the pair, defaulting to 2 s for cold-start pairs. Captures whether the image is already in the node's containerd cache; a first-pull penalty can dwarf the task itself on a 4 s workload.

Data locality (Channels A/B/C).

Channel A is KB-scale metadata threaded as env vars; Channel B is bulk output fetched over HTTP from the producer node's `ts-fileserver`; Channel C is the free same-node `hostPath` mount. The transfer term sums only the Channel-B edges, so the policy actively prefers to co-locate a child with its data-heavy parent.

Inter-node bandwidth.

Live measurement, every 10 min, fused with an EMA of *observed* initContainer fetches so a stale Wi-Fi number does not mislead the scheduler. A 100 MB transfer at 110 MB/s costs ≈0.9 s, comparable to a task's runtime variance, so the term is not a rounding error.

Ready time $t_{\mathsf{ready}}(n)$.

Honours placements already committed in this tick so two concurrent decisions do not over-pack one node; reduces to "now" on an idle node.

Thermal headroom and cooling class.

The scheduler discretises temperature into four buckets per cooling class (`cold`, `warm`, `hot`, `throttle`) and adds a soft penalty W_{hot} when headroom drops below θ_{hot}. The bucket also keys the profile store, so a hot run does not poison the cold-run median for the same template.

Failure rate with exponential decay.

Per (template, node) failure counter, half-life 14 h. A node that just dropped two Pods sees its score deducted by λ_{fail}; once the node has been quiet for a day the penalty is back to noise. Without decay one bad afternoon would permanently mark a node.

UCB exploration bonus.

$\sqrt{\ln N / n_{\tau,n}}$ keeps under-sampled pairs in contention so the policy does not lock onto the first node that happened to look good. Combined with a coverage-first phase (every feasible node sampled at least once before exploitation begins) this is the difference between converging to the right specialisation and getting stuck.

DAG upward rank.

The longest weighted path from a task to any sink; tasks with the largest rank are dispatched first so the critical path is considered early after priority and fairness have selected the workflow.

Workflow virtual runtime (vruntime).

CFS-style fairness across concurrently submitted workflows: the next workflow to dispatch from is the one whose accumulated CPU-share-weighted vruntime is smallest.

Avoids the starvation a strict priority queue would produce.

Gang membership.
A gang group is placed atomically or not at all, so MPI-style workloads do not deadlock waiting for the last rank.

Priority preemption with budgets.
A higher-priority Pending task may evict a running one, but only subject to per-node (≤ 3/min) and per-task (≤ 2 lifetime) caps and a near-completion immunity window. Prevents thrashing.

Defragmentation planner.
When a workflow is wedged by fragmented free capacity, a small migration plan is searched for; committed only if $\Delta J > \eta$. Avoids spinning a cluster on a single 4-CPU task that fits in aggregate but not contiguously.

Each term is calibrated in seconds (or a seconds-equivalent penalty), so the operator can read the score, attribute the verdict, and tune one term in isolation. The ablation experiments also strip each term in turn to quantify its individual contribution.

3.9 Baselines and why they fail on heterogeneous clusters

The simpler alternatives each fall short in a different way:

Default `kube-scheduler`.
Resource fit + affinity only. A 4 GiB memory-heavy task is placed on the first node that has 4 GiB free, regardless of whether the memory-optimised node sits idle next door.

Round-robin / random.
Oblivious to specialisation: a CPU-bound task lands on the memory-optimised node $1/k$ of the time, paying a 20 to 40 % per-task slowdown on our cluster.

Best-fit (resource-only).
Tends to fill the smallest matching node and leave specialised nodes under-utilised.

Static `nodeSelector`.
Brittle: breaks the moment a node fails, the topology changes, or two equally-good nodes exist.

Critical-path-only scheduling.
Correctly notices the important tasks inside one DAG, but ignores workflow priority, fairness, live failures, node temperature and new templates.

The proposed policy reduces to several of these as limit cases: with all λ zero and UCB disabled it is close to a pure earliest-completion-time DAG scheduler; with $\mu_{\tau,n,b}$ constant per node it is best-fit. Every term in J addresses a specific failure mode of a simpler policy.

3.10 Design-choice rationale

The major decisions, each justified by a concrete shortcoming of the alternative: *In-cluster controller + custom scheduler* replaces the prototype's laptop-side orchestration script, giving a `kubectl apply` UX; *Workflow + TaskTemplate CRDs* separate user from operator concerns; *producer-local HTTP data plane* collapses "where data lives" and "how fast it moves" into one path, removing the shared-NFS dependency; *pluggable policy interface* enables A/B testing without rebuilding images; *ECT + thermal-bucketed scoring* replaces the dimensionless weighted sum with an interpretable seconds-based score; *UCB + coverage-first exploration* samples every feasible node before exploiting, instead of pure greedy; *ConfigMap + PVC persistence* keeps learning across restarts; *Node-annotation telemetry* stays `kubectl`-visible.

4 IMPLEMENTATION JOURNEY

The system was built step by step. Each iteration fixed a problem that appeared in the previous version.

4.1 Iteration 0 to 1: simulator and legacy live scheduler

The first version was a simulator. It tested an 8-factor weighted scorer on synthetic workflows and a synthetic heterogeneous cluster. This gave the project its first data model, queue, readiness logic and profile store.

The next version ran on a single-host `kind` cluster. A local submitter created Pods stage by stage and passed parent outputs as environment variables. This proved that custom Pod binding and basic persistence worked, but it also showed the limits of the design: the DAG lived outside the cluster, the Pod spec was fixed too early, and there was no real cross-node data path.

4.2 Iteration 2: pluggable scheduling policy

The scheduler engine was separated from the placement policy. The engine collects the cluster state and ready tasks, while the policy decides whether to place, hold or preempt. This made it possible to keep the old scorer as a baseline and add the adaptive policy gradually.

4.3 Iteration 3: producer-local data plane

The shared host-path model was replaced. It worked on one machine, but not on two physical hosts. In the new model, each node keeps its own outputs and serves them over HTTP. If a child runs on the same node, it reads the local file. If it runs elsewhere, it fetches the file from the producer node. Bandwidth probing uses the same HTTP path.

4.4 Iteration 4: CRDs and controller

The local submitter was replaced with Kubernetes Custom Resources and an in-cluster controller. Users submit a `Workflow`; the controller reads the DAG, creates Pods for ready tasks, passes parent outputs to children and updates workflow status.

4.5 Iteration 5 to 6: telemetry and adaptive scheduling

Two telemetry DaemonSets were added. One measures bandwidth between nodes through the data plane. The other samples CPU temperature, or uses a load-based fallback when temperature is not exposed. The live scheduler was then switched from the legacy scorer to the adaptive policy. During this step, a temperature annotation mismatch was found and fixed.

4.6 Cross-cutting engineering

Multi-arch. The cluster uses both `amd64` and `arm64`. Images are built for both architectures and pushed to a local registry.
Persistence. Learned profiles are saved in Kubernetes storage and loaded again when the scheduler restarts. Older profile formats can be migrated instead of deleted.

5 RESEARCH METHODOLOGY

5.1 Two execution surfaces, one scheduler model

A working implementation is the prerequisite for any quantitative claim. Two execution surfaces are kept in sync and share the same scheduler model:

1. A **discrete-event simulator**. Deterministic, fast, and useful as a regression harness for placement behaviour.
2. A **real Kubernetes cluster** in two configurations: (a) single-host `kind` with four labelled workers (`ts-node-cpu-1/2`, `ts-node-mem-1`, `ts-node-io-1`), used for smoke tests and CI; (b) two-host `k3s`, provisioned and ready for the larger measurements planned next.

The validation is deliberately framed as *preliminary*: small pilot runs that confirm the system behaves as designed and that the comparisons against baselines point in the expected direction. The full quantitative evaluation (larger workloads, longer runs and confidence intervals on the two-host cluster) is the explicit subject of the next iteration.

5.2 Experimental topology

The two-host cluster spans two physically distinct machines on the same gigabit LAN, providing a real network hop (essential for the bandwidth signal) and two CPU architectures (essential to confirm the system works on heterogeneous hardware).

Role	Host	Arch	Logical k8s nodes
Master	Mac mini M2, 8 GB (macOS + Multipass)	arm64	`ts-master` (control plane + + registry, MEM_OPT, PASS
Worker	Laptop, Ryzen 7 PRO 8840HS, 64 GB (Windows + Multipass)	amd64	`ts-node-cpu-1`, `ts-node-c` (CPU_OPT), `ts-node-io-1` (IO_OPT), `ts-node-mem-1` (MEM_OPT)

Table 1: Two-host `k3s` topology: five logical nodes across two physical machines and two CPU architectures.

5.3 Workload and comparison protocol

The experiments use a project-defined workload mix rather than an external benchmark dataset. The task templates are CPU-bound (tight numeric loop), memory-bound (large buffer touches) and I/O-bound (file fetches through the data plane). DAG shapes range from linear pipelines to fan-out / fan-in topologies that exercise cross-node parent-to-child transfers.

Each experiment runs the same workload against the same cluster, with the only variable being the active policy: `adaptive`, `legacy`, `random`, `round-robin`, `best-fit`, and "no custom scheduler" (Pods left to `kube-scheduler`). Pairs are repeated $n = 10$ times; random seeds and probe schedules are pinned so every policy sees the same cluster state. Reproducibility is automated end-to-end by setup, build, verification and teardown scripts.

6 CONTRIBUTIONS

6.1 Theoretical

1. A *two-tier* scheduling formulation combining CFS-style workflow fairness (vruntime) with DAG-level task criticality (upward rank), so multiple workflows run concurrently without starvation and each still respects its critical path.
2. An *ECT estimator* computed against the *current* partial assignment, with a transfer term refined by live bandwidth annotations and an execution term backed by UCB-maintained priors.
3. A unified *scoring method* combining ECT with thermal and failure penalties and a UCB bonus; each term answers a specific failure mode of a simpler baseline.
4. A *thermal-bucketing scheme* that indexes profile observations by (cooling class, temperature), so the same task profile is meaningful across nodes with very different cooling.

6.2 Practical and engineering

1. A complete Kubernetes deployment: Workflow + TaskTemplate CRDs, in-cluster controller, custom scheduler and three DaemonSets, packaged as ordinary Kubernetes manifests. It coexists with any other workload.
2. A *producer-local data plane* (`ts-fileserver`) that sidesteps the need for a shared filesystem.
3. Two telemetry signals (`ts.io/bw-to-<peer>`, `ts.io/cpu-temp-c`) exposed as Node annotations rather than via a separate metrics stack, visible to any `kubectl` client.
4. Multi-arch container images (`linux/amd64`+`linux/arm64`) built through the container toolchain and served from a LAN-local registry.
5. A pluggable policy abstraction enabling adaptive versus legacy A/B testing through configuration.
6. A reproducible harness with environment configuration, setup scripts and end-to-end checks for node readiness, labels, DaemonSet coverage, telemetry annotations and workflow completion.
7. A simulator that runs the same scheduler model, giving cheap regression coverage.
8. Migration of the user surface from a laptop-resident submitter to CRD + controller, followed by a late-stage switch of the live scheduler from the legacy scorer to the adaptive policy. This also exposed and fixed a silent telemetry mismatch that had been dropping all temperature readings.

7 PERFORMANCE ANALYSIS METRICS

The evaluation uses the following metrics, computed per workflow and aggregated across $n = 10$ repeats of each (policy, workload) pair as mean $\pm$ 95 % CI:

Makespan M_w. Wall-clock time from the first Pod of w entering `Pending` to the last Pod reaching `Succeeded`. Primary headline metric.

Per-task slowdown $s_\tau = T_\tau / T_\tau^{\text{best}}$. Where T_τ^{best} is the time the same task achieves alone on its profile-matched node. Captures placement quality task-by-task.

Critical-path stretch. Observed makespan divided by the DAG's profile-matched critical-path length. Decouples policy quality from cluster capacity.

Cross-node ratio. Fraction of parent→child edges whose child ran on a different node than the parent. Lower is better when transfers are large; the policy should trade this off against ECT.

Scheduling latency and fairness. Pod `Pending`→`Scheduled` time (must not regress vs the default scheduler), and Jain's fairness index on per-workflow normalised makespan in multi-workflow runs.

8 PRELIMINARY RESULTS AND ANALYSIS

This chapter reports preliminary measurements from the single-host `kind` cluster used for development. The purpose is to confirm that (a) the live system behaves as designed, (b) the relative ordering of the baselines matches the expected behaviour, and (c) the architectural pivots improved observable outcomes. Numbers below are indicative; the full evaluation on the two-host `k3s` cluster follows in the next phase.

8.1 Pilot setup and end-to-end validation

Four worker nodes on a single host (`ts-node-cpu-1/2`, `ts-node-mem-1`, `ts-node-io-1`); workload is 10 instances of `linear-pipeline-1` (`task-io` $\rightarrow$ `task-mem` $\rightarrow$ `task-cpu`) submitted back-to-back; each (policy, workload) pair repeated 3 times.

The most important preliminary result is non-numerical: every component functions end-to-end. Workflows submitted via `kubectl apply` reach `FINISHED` and report per-task placements in `kubectl get workflows -o yaml`; `ts-scheduler` binds Pending Pods, `ts-controller` threads parent outputs into children, `ts-fileserver` serves outputs over HTTP, `ts-bw-probe` populates `ts.io/bw-to-<peer>` and `ts-thermal` populates `ts.io/cpu-temp-c`. ConfigMap profile state is correctly reloaded after a scheduler-pod restart (18 observations survived a manual rolling restart). In a representative ten-workflow run 8/10 finished successfully; the 2 failures both struck a single transiently-failing node and subsequent placements correctly routed away from it.

8.2 Specialisation behaviour

The clearest visible algorithmic effect is *task specialisation*: after ≈ 4 to 6 workflows the scheduler reliably steers `task-cpu` onto a CPU-optimised node, `task-mem` onto the memory-optimised one, and `task-io` onto the I/O-optimised one. The coverage-first exploration phase only needs one observation per feasible node before depth-phase takes over, and the simple 3-task DAG repeats the same templates on every workflow, so convergence is fast.

Task	Matching node (s)	Non-matching node (s)
`task-cpu`	6.0	8.1
`task-mem`	6.1	7.1
`task-io`	4.0	6.1

Table 2: Per-task runtime by node specialisation, observed on the `kind` pilot. Numbers are means of 8 successful runs; the gap is small because all four "nodes" share the same physical host, but the relative ordering matches the labels.

8.3 Indicative policy comparison

A small back-to-back comparison was run by flipping `TS_POLICY` (or `schedulerName`) between runs. Table 3 reports means of 3 runs of 10 workflows each, cold-started for each policy. The qualitative pattern matches the expected baseline behaviour: random and round-robin pay the specialisation tax constantly, the default scheduler is slightly better (resource fit), best-fit is better still, the legacy 8-factor scorer beats them all by using profile data, and the warm adaptive policy wins. The cold-start row for the adaptive policy is the price the policy explicitly pays for exploring every feasible node once before exploiting; by the $\approx 12^{\text{th}}$ workflow this cost is amortised.

Policy	Workflow makespan (s)	Cross-node ratio	Misplaced tasks (%)	
Random	24.3	0.71	51 %	High va
Round-robin	22.7	0.75	67 %	Determin
`kube-scheduler` (default)	21.1	0.66	48 %	No DAG
Best-fit	20.4	0.62	41 %	Resource-
Legacy 8-factor	17.2	0.41	22 %	Static
Adaptive (cold)	21.6	0.58	38 %	Still in
Adaptive (warm, $\geq$12 wf)	14.8	0.18	9 %	Learn

Table 3: Indicative pilot comparison on the `kind` cluster. "Misplaced" means a task ran on a node whose declared type does not match the task's class.

9 OPERATOR'S GUIDE AND CONVERGENCE EXPECTATIONS

This short chapter documents what an operator must know to deploy TaskScheduler and run their own workloads on it. It complements the reference `README.md` in the repository.

9.1 Prerequisites and installation

A Kubernetes cluster 1.27+ (`k3s` and `kind` have both been validated; vanilla `kubeadm` should work but is untested). Each worker node must carry `node-type=CPU_OPT | MEM_OPT | IO_OPT | GENERAL`; optional cooling labels are `ts.cooling/class=PASSIVE | STANDARD | HIGH | EXTREME` and `ts.cooling/throttle-c=<celsius>`. Each node must allow host-path mounts for the data plane and thermal probe. Install the supplied Kubernetes manifests; CRDs, RBAC, controller, scheduler and the three DaemonSets land in namespace `ts-system`.

9.2 Task templates and workflows

Operators register each task image once as a `TaskTemplate`; users submit `Workflows` referencing those templates and observe progress via `kubectl get workflows -w`. A minimal example of each:

```
apiVersion: ts.io/v1
kind: TaskTemplate
metadata: {name: cpu-cruncher}
spec:
  image: registry.lan:5000/ts-task-cpu:v1
  taskClass: CPU_BOUND
  cpuRequest: 1.0
  memoryRequestMi: 256
  compatibleNodeTypes: [CPU_OPT, GENERAL]
  expectedOutputBytes: 1024
Second object:
apiVersion: ts.io/v1
kind: Workflow
metadata: {name: my-pipeline}
spec:
  priority: BATCH
```

```
tasks:
  - {name: ingest,  template: io-fetch}
  - name: process
    template: mem-process
    dependsOn:
      - {parent: ingest, dependencyType: DATA, dataFields: [generated_file_path]}
  - name: report
    template: cpu-cruncher
    dependsOn:
      - {parent: process, dependencyType: DATA}
```

Tasks that produce outputs consumed downstream must print one line to stdout of the form `__TS_OUTPUT__={"key":"value", ...}`. The controller parses the JSON and injects each key as `TS_PARENT_<NAME>_<KEY>` on the child Pod; for file outputs the conventional key is `generated_file_path` and the child also receives `TS_PARENT_<NAME>_FILESERVER_URL` to `curl` the file when the parent ran on a different node.

9.3 Convergence and remediation

For each `TaskTemplate` the scheduler walks through coverage (sample every feasible node, $\approx k$ runs), depth (3 observations per node, $\approx 3k$ runs) and normal (exploit + 10 % random exploration + UCB re-exploration). On a typical 4 to 6 node home/SMB cluster a recurring template becomes *warm* within ≈ 12 to 18 successful executions, the cost of a coffee break rather than a training run. Cold-start placements are not random: they use the `compatibleNodeTypes` prior and the default-runtime constants in the ECT estimator.

10 CONCLUSION AND FURTHER WORK

10.1 Summary

This project built a custom scheduler for Kubernetes workflows. It is meant for small heterogeneous clusters, where nodes can differ in CPU type, memory, storage, architecture, temperature and network position.

The final system uses two Custom Resources, an in-cluster controller, a custom scheduler and three DaemonSets. It can run as a normal Kubernetes deployment. Users submit workflows through YAML, while the controller creates the Pods and the scheduler chooses where they run.

The project started as a simulator and then moved to a real Kubernetes prototype. Along the way, the design added a local data plane, bandwidth probing, thermal telemetry, learned task profiles and an adaptive placement policy. The pilot results show that the components work together and that the scheduler learns useful task to node preferences. The larger two-host evaluation remains the next step.

10.2 Further work

- **Full evaluation on the two-host cluster.** The topology is ready; the same workloads need to be run with proper confidence intervals against each baseline.
- **External workflow benchmarks.** Pegasus DAX, CyberShake or similar benchmark DAGs should be added after the current Kubernetes-native validation is complete, so the system can be compared against published workflow-scheduling results.
- **Live preemption / checkpoint-restore.** The model leaves the slots open and the simulator-side planner exists; wiring it into the live binder is the next step.
- **Speculative execution** for stragglers, triggered by runtime-percentile thresholds from the profile store.
- **Energy and cost-aware scoring** as an extra term inside the same J framework, once node-level metering is available.
- **Learned ECT estimators.** Replacing the profile-based execution priors with a small online regressor over task features (image digest, resource request, parent template) would close the cold-start gap.

BIBLIOGRAPHY

[1] H. Topcuoglu, S. Hariri, and M.-Y. Wu, *Performance-effective and low-complexity task scheduling for heterogeneous computing*, IEEE Trans. on Parallel and Distributed Systems, 13(3):260 to 274, 2002.

[2] P. Auer, N. Cesa-Bianchi, and P. Fischer, *Finite-time analysis of the multiarmed bandit problem*, Machine Learning, 47(2 to 3):235 to 256, 2002.

[3] A. Verma, L. Pedrosa, M. Korupolu, D. Oppenheimer, E. Tune, and J. Wilkes, *Large-scale cluster management at Google with Borg*, Proc. EuroSys, 2015.

[4] M. Schwarzkopf, A. Konwinski, M. Abd-El-Malek, and J. Wilkes, *Omega: flexible, scalable schedulers for large compute clusters*, Proc. EuroSys, 2013. ke

[5] T. Li, Z. Xu, J. Tang, and Y. Wang, *Model-Free Control for Distributed Stream Data Processing using Deep Reinforcement Learning*, Proc. VLDB Endowment, 11(6):705 to 718, 2018.

[6] The Kubernetes Authors, *Kubernetes documentation*, `https://kubernetes.io/docs/`.

[7] Rancher Labs, *k3s: Lightweight Kubernetes*, `https://k3s.io/`.